

FleetLogix - Sistema de Gestión de Transporte y Logística

Guía General del Proyecto - Documentación Completa

Jacquet Alexis

Científico de Datos

Octubre 2025

Abstract

FleetLogix es un sistema integral de gestión de transporte y logística desarrollado con metodología científica de datos. Este documento presenta la arquitectura completa del proyecto, abarcando desde la generación de 505,650+ registros sintéticos hasta la implementación de un Data Warehouse en Snowflake y arquitectura cloud en AWS.

Contents

1	RESUMEN EJECUTIVO	3
1.1	Objetivos Cumplidos	3
1.2	Cifras Clave	3
1.3	Tecnologías Utilizadas	3
1.4	Entregables	3
2	Guía General del Proyecto	5
2.1	Resumen Ejecutivo del Proyecto	5
2.1.1	Objetivos del Proyecto	5
2.1.2	Cifras Clave	5
2.2	Arquitectura del Proyecto	6
2.2.1	Capa 1: Base de Datos Operacional (PostgreSQL)	6
2.2.2	Capa 2: Data Warehouse (Snowflake)	6
2.2.3	Capa 3: Arquitectura AWS	6
2.3	Estructura del Proyecto	7
2.4	Tecnologías Utilizadas	8
2.4.1	Base de Datos	8
2.4.2	Lenguajes de Programación	8
2.4.3	Librerías Python Principales	8
2.4.4	Servicios AWS	8
2.5	Avances del Proyecto	9
2.5.1	Avance 1: Generación de Datos Sintéticos	9
2.5.2	Avance 2: Análisis y Optimización SQL	10
2.5.3	Avance 3: Data Warehouse y ETL	11
2.5.4	Avance 4: Arquitectura AWS	12
2.6	Instalación y Configuración	13
2.6.1	Requisitos Previos	13
2.6.2	Paso 1: Clonar repositorio	13
2.6.3	Paso 2: Crear entorno virtual	13
2.6.4	Paso 3: Instalar dependencias	13
2.6.5	Paso 4: Configurar variables de entorno	13
2.6.6	Paso 5: Crear base de datos PostgreSQL	14
2.7	Uso de Scripts	15
2.7.1	1. Generación de Datos	15
2.7.2	2. Análisis SQL	15
2.7.3	3. ETL a Snowflake	15

2.7.4	4. Configuración AWS	15
2.8	Resultados y Métricas	16
2.8.1	Validación de Datos	16
2.8.2	Performance de Queries	16
2.8.3	Métricas de Negocio	16
2.9	Referencias	17
2.9.1	Documentación Técnica	17
2.9.2	Metodología	17
2.9.3	Herramientas	17
2.10	Autor y Licencia	18
2.10.1	Autor	18
2.10.2	Licencia	18
2.10.3	Agradecimientos	18

Chapter 1

RESUMEN EJECUTIVO

1.1 Objetivos Cumplidos

- Generación de **505,650+** registros de datos sintéticos coherentes y realistas
- Análisis y optimización de **12 queries SQL** con mejoras de 20-80%
- Implementación de **modelo dimensional tipo estrella** en Snowflake
- Despliegue de **arquitectura AWS serverless** escalable y resiliente
- Pipeline ETL completo con transformaciones científicas

1.2 Cifras Clave

Métrica	Valor	Observación
Registros totales	505,650+	100% coherencia referencial
Queries optimizadas	12	Mejora promedio 65%
Índices estratégicos	5	Cobertura queries críticas
Dimensiones DW	6	Modelo estrella completo
Tablas de hechos	2	Trips y Deliveries
Funciones Lambda	3	Procesamiento serverless
Tiempo ETL	~15 min	505k registros procesados

1.3 Tecnologías Utilizadas

- **PostgreSQL 15+**: Base de datos transaccional OLTP con 505k+ registros
- **Snowflake**: Data Warehouse cloud con modelo dimensional
- **Python 3.9+**: Scripts ETL, generación de datos y automatización
- **AWS**: RDS, Lambda, S3, DynamoDB, SNS para arquitectura cloud
- **Pandas/NumPy**: Procesamiento y análisis científico de datos
- **Faker**: Generación de datos sintéticos realistas

1.4 Entregables

1. **Scripts Python** (7 archivos): Generación de datos, ETL, AWS setup

2. **Scripts SQL** (4 archivos): Queries, índices, modelo dimensional
 3. **Documentación** (5 documentos): Análisis completo de cada avance
 4. **Base de datos** operacional con 505,650 registros validados
 5. **Data Warehouse** en Snowflake con modelo estrella
-

Chapter 2

Guía General del Proyecto

2.1 Resumen Ejecutivo del Proyecto

FleetLogix es un sistema integral de gestión de transporte y logística diseñado con metodología científica de datos. El proyecto abarca desde la generación de datos sintéticos hasta la implementación de un Data Warehouse en la nube, pasando por optimización de consultas y arquitectura AWS.

2.1.1 Objetivos del Proyecto

- **Avance 1:** Generación de 505,650+ registros de datos sintéticos coherentes
- **Avance 2:** Análisis y optimización de 12 queries SQL críticas
- **Avance 3:** Implementación de modelo dimensional tipo estrella en Snowflake
- **Avance 4:** Arquitectura AWS escalable con Lambda, RDS y S3

2.1.2 Cifras Clave

Métrica	Valor
Registros totales generados	505,650+
Tablas maestras	650 registros
Tablas transaccionales	505,000 registros
Queries optimizadas	12
Mejora de performance	20-80%
Índices estratégicos	5
Dimensiones en DW	6
Tablas de hechos	2
Funciones Lambda	3

2.2 Arquitectura del Proyecto

2.2.1 Capa 1: Base de Datos Operacional (PostgreSQL)

POSTGRESQL - BD TRANSACCIONAL

Tablas Maestras:

- vehicles (200 registros)
- drivers (400 registros)
- routes (50 registros)

Tablas Transaccionales:

- trips (100,000 registros)
- deliveries (400,000 registros)
- maintenance (5,000 registros)

2.2.2 Capa 2: Data Warehouse (Snowflake)

SNOWFLAKE - DATA WAREHOUSE

Modelo Estrella:

Dimensiones:

- dim_date
- dim_time
- dim_vehicle
- dim_driver
- dim_route
- dim_customer

Tablas de Hechos:

- fact_trips
- fact_deliveries

2.2.3 Capa 3: Arquitectura AWS

AWS CLOUD SERVICES

- RDS PostgreSQL (BD Principal)
- S3 (Almacenamiento datos históricos)
- Lambda (Procesamiento serverless)
- DynamoDB (Cache entregas)
- SNS (Notificaciones)

2.3 Estructura del Proyecto

FleetLogix/

Scripts/

01_data_generation.py	#	Generación 505k+ registros
02_queries_analysis.sql	#	12 queries SQL analizadas
03_optimization_indexes.sql	#	5 índices optimizados
04_dimensional_model.sql	#	DDL Data Warehouse
05_etl_pipeline.py	#	Pipeline ETL completo
06_aws_setup.py	#	Configuración AWS
lambda_functions.py	#	3 funciones Lambda

Documentación/

README.md	#	Este documento
Análisis_del_modelo_proporcionado.md		
Manual_Consultas_SQL.md		
Análisis_Snowflake_ETL.md		
AWS_Análisis_Arquitectura.md		

Datos/

(Generados por scripts)

2.4 Tecnologías Utilizadas

2.4.1 Base de Datos

- **PostgreSQL 15+**: Base de datos relacional principal
- **Snowflake**: Data Warehouse en la nube

2.4.2 Lenguajes de Programación

- **Python 3.9+**: Scripts ETL y generación de datos
- **SQL**: Consultas y definición de esquemas

2.4.3 Librerías Python Principales

Generación de datos

faker==20.1.0

pandas==2.1.3

numpy==1.26.2

Conexión a BD

psycopg2-binary==2.9.9

snowflake-connector-python==3.5.0

AWS

boto3==1.29.7

Visualización y logs

tqdm==4.66.1

python-dotenv==1.0.0

2.4.4 Servicios AWS

- **RDS**: PostgreSQL gestionado
- **S3**: Almacenamiento de objetos
- **Lambda**: Funciones serverless
- **DynamoDB**: Base de datos NoSQL
- **SNS**: Servicio de notificaciones

2.5 Avances del Proyecto

2.5.1 Avance 1: Generación de Datos Sintéticos

Script: 01_data_generation.py

Objetivo: Generar 505,650+ registros de datos coherentes y realistas para FleetLogix.

Características: - Nombres españoles con consistencia de género garantizada - Distribuciones horarias realistas para logística (picos 8-10am, 2-4pm) - Validaciones exhaustivas de integridad referencial - Control de calidad científico completo - Feedback visual con tqdm y logs estructurados

Registros generados:

Tablas Maestras (650 registros):

- vehicles: 200
- drivers: 400
- routes: 50

Tablas Transaccionales (505,000 registros):

- trips: 100,000
- deliveries: 400,000
- maintenance: 5,000

TOTAL: 505,650 registros

Ejecución:

python Scripts/01_data_generation.py

2.5.2 Avance 2: Análisis y Optimización SQL

Scripts: - 02_queries_analysis.sql: 12 queries de negocio - 03_optimization_indexes.sql: 5 índices estratégicos

Objetivo: Optimizar las consultas más críticas del sistema mediante análisis científico.

Queries clasificadas por complejidad:

Básicas (3 queries): 1. Inventario de flota por tipo y estado 2. Conductores con certificaciones próximas a vencer 3. Resumen operacional de viajes por estado

Intermedias (5 queries): 4. Análisis de demanda geográfica por ciudad destino 5. Performance de conductores con métricas de eficiencia 6. Viajes completados en último trimestre con análisis temporal 7. Entregas pendientes agrupadas por fecha programada 8. Top 10 rutas más utilizadas con indicadores de rentabilidad

Complejas (4 queries): 9. Dashboard ejecutivo integrado con KPIs críticos 10. Análisis de eficiencia de combustible por tipo de vehículo 11. Ranking de conductores por tasa de entregas exitosas 12. Análisis predictivo de mantenimiento preventivo

Mejoras de Performance: | Índice | Mejora | Queries beneficiadas | |———|———|———|
—| | idx_trips_route_departure | 75-80% | 4, 6, 8, 10 | | idx_trips_status_datetime | 60-70% | 5, 9, 11 | | idx_deliveries_scheduled_status | 65-75% | 7, 12 | | idx_trips_deliveries | 70-80% | 4, 9, 11 | | idx_maintenance_vehicle_date | 50-60% | 9, 12 |

2.5.3 Avance 3: Data Warehouse y ETL

Scripts: - 04_dimensional_model.sql: Definición del modelo dimensional - 05_etl_pipeline.py: Pipeline ETL completo

Objetivo: Implementar un Data Warehouse tipo estrella en Snowflake para análisis OLAP.

Modelo Dimensional:

Dimensiones (6): 1. **dim_date:** Dimensión temporal con análisis fiscal 2. **dim_time:** Análisis por hora del día y turnos 3. **dim_vehicle:** Historial de vehículos (SCD Type 2) 4. **dim_driver:** Historial de conductores (SCD Type 2) 5. **dim_route:** Clasificación de rutas por dificultad 6. **dim_customer:** Segmentación de clientes

Tablas de Hechos (2): 1. **fact_trips:** Viajes con métricas agregadas 2. **fact_deliveries:** Entregas con SLA y satisfacción

Pipeline ETL: - **Extracción:** PostgreSQL con validaciones estadísticas - **Transformación:** Cálculos basados en datos reales - **Carga:** Snowflake con validación de integridad - **Validación:** Tests automáticos de calidad

Columnas calculadas científicas:

```
# Basadas en análisis estadístico real
performance_category    # Success rate + experiencia
difficulty_level         # Varianza duración + distancia
route_type              # Urbana/Interurbana/Rural
customer_type           # Empresa/Individual por volumen
customer_category       # Premium/Regular/Ocasional
```

2.5.4 Avance 4: Arquitectura AWS

Scripts: - `06_aws_setup.py`: Configuración infraestructura - `lambda_functions.py`: Funciones serverless

Objetivo: Desplegar una arquitectura escalable y resiliente en AWS.

Componentes:

1. RDS PostgreSQL: - Instancia: db.t3.micro (Free tier) - Engine: PostgreSQL 15.4 - Storage: 20GB GP2 - Backups automáticos: 7 días - Multi-AZ: No (desarrollo)

2. S3 Bucket: - Nombre: fleetlogix-data - Estructura: - `raw-data/`: Datos crudos - `processed-data/`: Datos procesados - `backups/`: Respaldos - `logs/`: Registros de actividad - Lifecycle: Archivo a Glacier después de 90 días

3. DynamoDB: - Tabla: deliveries_status - Partition key: delivery_id - TTL: 30 días - Capacidad: On-demand

4. Lambda Functions (3):

a) lambda_verificar_entrega: - Trigger: API Gateway - Runtime: Python 3.11 - Timeout: 10s - Memoria: 256MB - Función: Verificar estado de entrega en DynamoDB

b) lambda_calcular_eta: - Trigger: EventBridge (cada 5 min) - Runtime: Python 3.11 - Timeout: 15s - Memoria: 512MB - Función: Calcular tiempo estimado de llegada

c) lambda_alertas_entregas: - Trigger: DynamoDB Streams - Runtime: Python 3.11 - Timeout: 30s - Memoria: 256MB - Función: Enviar alertas vía SNS por entregas retrasadas

5. SNS Topics: - `alertas-entregas-retrasadas` - `notificaciones-mantenimiento` - `reportes-diarios`

2.6 Instalación y Configuración

2.6.1 Requisitos Previos

1. Python 3.9+

```
python --version
```

2. PostgreSQL 15+

```
psql --version
```

3. Cuenta Snowflake (trial gratuita disponible)
4. Cuenta AWS (Free tier disponible)

2.6.2 Paso 1: Clonar repositorio

```
cd C:\Users\Usuario\Desktop\HENRY\MODULO 2
cd documentacion
```

2.6.3 Paso 2: Crear entorno virtual

```
python -m venv venv
.\venv\Scripts\activate # Windows
```

2.6.4 Paso 3: Instalar dependencias

```
pip install -r requirements.txt
```

2.6.5 Paso 4: Configurar variables de entorno

Crear archivo .env:

```
# PostgreSQL
POSTGRES_HOST=localhost
POSTGRES_DB=fleetlogix
POSTGRES_USER=fleetlogix_user
POSTGRES_PASSWORD=fleetlogix123
POSTGRES_PORT=5432

# Snowflake
SNOWFLAKE_USER=tu_usuario
SNOWFLAKE_PASSWORD=tu_password
SNOWFLAKE_ACCOUNT=tu_cuenta
SNOWFLAKE_WAREHOUSE=FLEETLOGIX_WH
SNOWFLAKE_DATABASE=FLEETLOGIX_DW
SNOWFLAKE_SCHEMA=ANALYTICS

# AWS
AWS_ACCESS_KEY_ID=tu_access_key
AWS_SECRET_ACCESS_KEY=tu_secret_key
AWS_REGION=us-east-1
```

2.6.6 Paso 5: Crear base de datos PostgreSQL

```
-- Crear usuario y BD
CREATE USER fleetlogix_user WITH PASSWORD 'fleetlogix123';
CREATE DATABASE fleetlogix OWNER fleetlogix_user;
GRANT ALL PRIVILEGES ON DATABASE fleetlogix TO fleetlogix_user;

-- Conectar a BD
\c fleetlogix

-- Ejecutar schema
\i fleetlogix_db_schema.sql
```

2.7 Uso de Scripts

2.7.1 1. Generación de Datos

```
# Generar 505,650+ registros  
python Scripts/01_data_generation.py
```

```
# Salida esperada:  
# 200 vehículos creados  
# 400 conductores creados  
# 50 rutas creadas  
# 100,000 viajes creados  
# 400,000 entregas creadas  
# 5,000 mantenimientos creados  
# TOTAL: 505,650 registros
```

2.7.2 2. Análisis SQL

```
# Conectar a PostgreSQL  
psql -U fleetlogix_user -d fleetlogix
```

```
# Ejecutar queries  
\i Scripts/02_queries_analysis.sql
```

```
# Ejecutar optimizaciones  
\i Scripts/03_optimization_indexes.sql
```

2.7.3 3. ETL a Snowflake

```
# Ejecutar pipeline ETL  
python Scripts/05_etl_pipeline.py
```

```
# Salida esperada:  
# Dimensiones cargadas  
# Hechos cargados  
# Validaciones exitosas
```

2.7.4 4. Configuración AWS

```
# Configurar infraestructura  
python Scripts/06_aws_setup.py
```

```
# Desplegar funciones Lambda  
python Scripts/lambda_functions.py
```

2.8 Resultados y Métricas

2.8.1 Validación de Datos

```
# Estadísticas generadas
{
  "total_registros": 505650,
  "integridad_referencial": "100%",
  "registros_duplicados": 0,
  "valores_nulos_criticos": 0,
  "coherencia_temporal": "100%",
  "distribucion_geografica": "Balanceada"
}
```

2.8.2 Performance de Queries

Query	Antes	Después	Mejora
Q4: Análisis geográfico	180ms	45ms	75%
Q9: Dashboard ejecutivo	350ms	85ms	76%
Q11: Ranking conductores	420ms	95ms	77%

2.8.3 Métricas de Negocio

KPIs FleetLogix:

Utilización de flota: 78.5%
Tasa de entregas exitosas: 95.2%
Eficiencia de combustible: 12.3 km/L
Tiempo promedio de entrega: 2.4 días
Costo por km: \$2.45
Satisfacción cliente: 4.5/5.0

2.9 Referencias

2.9.1 Documentación Técnica

1. **PostgreSQL:** <https://www.postgresql.org/docs/>
2. **Snowflake:** <https://docs.snowflake.com/>
3. **AWS RDS:** <https://docs.aws.amazon.com/rds/>
4. **AWS Lambda:** <https://docs.aws.amazon.com/lambda/>

2.9.2 Metodología

- Kimball, R. (2013). *The Data Warehouse Toolkit*
- Date, C.J. (2004). *An Introduction to Database Systems*
- Inmon, W.H. (2005). *Building the Data Warehouse*

2.9.3 Herramientas

- **Python Faker:** <https://faker.readthedocs.io/>
- **Pandas:** <https://pandas.pydata.org/>
- **Boto3:** <https://boto3.amazonaws.com/>

2.10 Autor y Licencia

2.10.1 Autor

Jacquet Alexis
Científico de Datos
HENRY - Módulo 2
Octubre 2025

2.10.2 Licencia

Este proyecto es parte del programa académico de HENRY y está destinado únicamente para fines educativos.

2.10.3 Agradecimientos

- Equipo docente de HENRY
- Comunidad de Python y PostgreSQL
- Documentación oficial de AWS y Snowflake

Documento: README - Guía General del Proyecto FleetLogix

Autor: Jacquet Alexis - Científico de Datos

Institución: HENRY - Módulo 2

Fecha: Octubre 2025

Contacto: [Disponible bajo solicitud]